Learn *C/C++* Programming
Learn *C#* Programming
Learn *Java* Programming
Learn *Pascal* Programming
Learn *Web Development*
General Computer Articles
Links
Contact

Learn C programming tutorial lesson 9 - Structures

We already know that arrays are many variables of the same type grouped together under the same name. Structures are like arrays except that they allow many variables of different types grouped together under the same name. For example you can create a structure called person which is made up of a string for the name and an integer for the age. Here is how you would create that person structure in C:

```
struct person
{
   char *name;
   int age;
};
```

The above is just a declaration of a type. You must still create a variable of that type to be able to use it. Here is how you create a variable called p of the type person:

```
#include<stdio.h>

struct person
{
   char *name;
   int age;
};

int main()
{
   struct person p;
   return 0;
}
```

To access the string or integer of the structure you must use a dot between the structure name and the variable name.

```
#include<stdio.h>

struct person
{
   char *name;
   int age;
};

int main()
{
```

```
   struct person p;
   p.name = "John Smith";
   p.age = 25;
   printf("%s",p.name);
   printf("%d",p.age);
   return 0;
}
```

Type definitions

You can give your own name to a variable using a type definition. Here is an example of how to create a type definition called intptr for a pointer to an integer.

```
#include<stdio.h>

typedef int *intptr;

int main()
{
   intptr ip;
   return 0;
}
```

Type definitions for a structure

If you don't like to use the word struct when declaring a structure variable then you can create a type definition for the structure. The name of the type definition of a structure is usually all in uppercase letters.

```
#include<stdio.h>

typedef struct person
{
   char *name;
   int age;
} PERSON;

int main()
{
   PERSON p;
   p.name = "John Smith";
   p.age = 25;
   printf("%s",p.name);
   printf("%d",p.age);
   return 0;
}
```

Pointers to structures

When you use a pointer to a structure you must use -> instead of a dot.

```
#include<stdio.h>

typedef struct person
```

```
{
   char *name;
   int age;
} PERSON;

int main()
{
   PERSON p;
   PERSON *pptr;
   PERSON pptr = &p;
   pptr->name = "John Smith";
   pptr->age = 25;
   printf("%s",pptr->name);
   printf("%d",pptr->age);
   return 0;
}
```

Unions

Unions are like structures except they use less memory. If you create a structure with a double that is 8 bytes and an integer that is 4 bytes then the total size will still only be 8 bytes. This is because they are on top of each other. If you change the value of the double then the value of the integer will change and if you change the value of the integer then the value of the double will change. Here is an example of how to declare and use a union:

```
#include<stdio.h>

typedef union num
{
   double d;
   int i;
} NUM;

int main()
{
   NUM n;
   n.d = 3.14;
   n.i = 5;
   return 0;
}
```